

AI-driven Polygonal Image Stylization: Bridging Algorithmic Art and Machine Learning Design

DES646 Mid-Term Project Report

Group Members:

Name	Roll No	Email
Banoth Anand	220281	banotha22@iitk.ac.in
Lavudya Nethaji	220584	lnethaji22@iitk.ac.in
N Ravi	220675	nravi23@iitk.ac.in

November 8, 2025

Abstract

This project presents a comprehensive system for transforming natural images into artistic polygonal mosaics using two complementary approaches: algorithmic stylization through Delaunay triangulation and adaptive quadtree methods, and learned stylization using Pix2Pix GAN. The system provides an interactive Streamlit interface that enables users to experiment with different stylization modes, compare results, and generate artistic visualizations in real-time. The implementation successfully bridges traditional computational geometry with modern deep learning techniques, demonstrating the significant potential of artificial intelligence in creative design applications. By integrating both algorithmic precision and learned aesthetics, the system offers flexible and powerful tools for artistic image transformation.

1 Introduction

1.1 Background

Polygonal image stylization represents a sophisticated technique that transforms ordinary photographs into simplified geometric representations, creating visually striking artistic effects. This computational process systematically reduces visual complexity while preserving essential structural and semantic information. Traditional approaches in this domain primarily rely on computational geometry algorithms, while modern methods increasingly leverage deep learning architectures to learn complex stylization patterns directly from data, offering new possibilities for creative expression.

1.2 Problem Statement

Creating compelling polygonal stylizations requires carefully balancing artistic quality with computational efficiency. Algorithmic methods provide excellent control and interpretability but may lack the nuanced understanding of visual aesthetics that human artists possess. Conversely, learned methods can capture complex visual patterns and artistic styles but typically require substantial training data and computational resources. This project addresses the fundamental challenge of integrating both approaches into a unified, user-friendly system that leverages the strengths of each methodology while mitigating their respective limitations.

1.3 Objectives

The primary objectives of this project encompass several key areas. First, we aim to implement geometric stylization through Delaunay triangulation and adaptive quadtree algorithms, providing robust algorithmic foundations for polygonal transformation. Second, we plan to train a Pix2Pix model incorporating U-Net architecture with PatchGAN discriminator for advanced photo-to-polygon style transfer capabilities. Third, we will build an interactive user interface using Streamlit to facilitate visualization and comparison between algorithmic and AI-based outputs. Additionally, we intend to explore perceptual and aesthetic evaluations of stylized results to understand the qualitative aspects of our transformations. Finally, we commit to delivering a fully functional prototype demonstrating real-time stylization capabilities on custom images.

2 Methodology

2.1 System Architecture

The system architecture comprises three main components that work in concert to deliver comprehensive stylization capabilities. The Algorithmic Stylization Module implements Delaunay triangulation and quadtree-based rectangular partitioning, providing traditional computational geometry approaches. The Deep Learning Module incorporates a Pix2Pix model specifically designed for learned stylization, leveraging modern neural network architectures. The User Interface Module, built on Streamlit, offers an interactive application framework that enables users to engage with the system intuitively and effectively.

2.2 Data Preparation Pipeline

The data preparation pipeline, implemented in `data/prepare_dataset.py`, executes a systematic sequence of operations. The process begins with image collection, offering two distinct options: utilizing the OpenImages dataset accessed through FiftyOne for up to 100 images, or employing custom images from a local directory for specialized applications. Target generation follows, applying triangular or rectangular stylization to each source image while generating targets at multiple resolutions including 256×256 and 512×512 pixels. The pipeline incorporates letter-boxing techniques to maintain aspect ratios while creating standardized square outputs. Finally, dataset organization ensures systematic storage, with source images residing in `data/images/` and stylized targets stored in structured directories like `data/targets/triangles_256/` or `data/targets/rectangles_256/`, with careful filename matching between sources and targets.

2.3 Training Methodology

The Pix2Pix training process follows a meticulously designed methodology. Data loading involves processing paired datasets containing source photographs and their corresponding polygonal targets. Data augmentation strategies include horizontal flipping with 50% probability, rotation within ± 10 degrees, and comprehensive color jitter encompassing brightness, contrast, saturation, and hue adjustments, applied exclusively to source images while preserving target integrity. The loss function framework incorporates multiple components: adversarial loss through PatchGAN discriminator using binary cross-entropy, pixel-wise reconstruction loss via L1 metric with weighting factor $\lambda = 100.0$, and perceptual loss utilizing VGG feature matching with $\lambda = 10.0$. Training configuration specifies 512×512 image size, batch size of 2, learning rate of 0.0002, training duration of 10 epochs, Adam optimizer with $\beta_1 = 0.5$ and $\beta_2 = 0.999$ parameters, and an 80/20 train/validation split for robust model evaluation.

2.4 Inference Pipeline

The inference module, implemented in `inference/infer.py`, provides comprehensive processing capabilities supporting single image processing, batch folder processing, flexible image sizing from 256 to 1024 pixels, and automatic device selection prioritizing CUDA with CPU fallback, ensuring broad compatibility across different hardware configurations.

3 Implementation Details

3.1 Algorithmic Stylization

The triangular stylization process, implemented in `tools/triangulate.py`, encompasses several sophisticated components. Point sampling employs multiple strategies: edge-based sampling detects edges using Sobel operators and samples points from high-gradient regions; random sampling adds stochastic points for uniform coverage; grid sampling creates regular patterns with controlled jitter for spatial consistency; and border point placement ensures comprehensive boundary representation. Triangulation utilizes SciPy’s Delaunay implementation as the primary method, with OpenCV’s Subdiv2D serving as a reliable fallback for compatibility, generating coherent triangle meshes from sampled points. Rendering involves color assignment through mean or centroid-based approaches per triangle, with optional edge drawing for enhanced visual definition. Key parameters include point count (defaulting to 800), edge threshold for detection sensitivity ranging 1–255 (default 50), and color mode selection between mean and centroid methods.

The rectangular stylization employs recursive quadtree decomposition through an adaptive algorithm that begins with the entire image as the root rectangle, calculates regional variance, and recursively subdivides into four quadrants when variance exceeds threshold and size surpasses minimum dimensions, otherwise assigning mean color and terminating recursion. Critical parameters include minimum block size (default 8 pixels) and variance threshold for splitting (default 12.0). This approach offers significant advantages through adaptive detail focusing on high-variance regions, creation of natural rectangular mosaics, and computational efficiency.

3.2 Deep Learning Module

The U-Net generator architecture, implemented in `models/pix2pix_train.py`, features a sophisticated encoder-decoder structure. The encoder component employs seven downsampling blocks, each comprising Conv2d, BatchNorm, and LeakyReLU operations, with feature channels progressively expanding from 64 through 128, 256, 512, culminating in a 512→512 final bottleneck. The decoder counterpart utilizes seven upsampling blocks featuring ConvTranspose2d, BatchNorm, and ReLU operations, enhanced with skip connections from corresponding encoder layers, dropout regularization in initial decoder blocks, and final Tanh activation for normalized output ranging $[-1, 1]$. Key architectural characteristics include skip connections for fine detail preservation, symmetric encoder-decoder organization, batch normalization for training stability, and Tanh activation for output normalization.

The PatchGAN discriminator processes concatenated source and target/generated images across six channels through four convolutional layers, producing patch-based classification maps rather than single scalar outputs, with 70×70 patch receptive fields for 256×256 images. This architecture provides distinct advantages including enhanced efficiency compared to full-image discriminators, promotion of local realism, and accelerated training and inference processes.

Perceptual loss implementation utilizes pre-trained VGG16 with ImageNet weights, extracting features from conv1_2, conv2_2, and conv3_3 layers, computing L1 loss between feature maps while maintaining VGG network freezing during training without gradient updates. This approach serves to capture high-level semantic similarity, deliver superior perceptual quality

compared to pixel-wise metrics, and encourage realistic texture and structure generation.

The training loop orchestrates generator training through forward passes, adversarial loss computation from discriminator feedback, L1 reconstruction loss calculation, perceptual loss derivation from VGG features, and combined loss optimization integrating $L_{adv} + \lambda_1 L_{L1} + \lambda_2 L_{perc}$. Discriminator training involves forward passes with real (source, target) and fake (source, generated) pairs, binary cross-entropy loss computation for real/fake classification, and discriminator weight updates. Training strategy employs alternating generator and discriminator updates with 0.0002 learning rate through Adam optimizer, supplemented by batch normalization and dropout for effective regularization.

3.3 Data Processing Utilities

The utility functions in `utils/image_ops.py` provide essential image processing capabilities including image reading through OpenCV with BGR to RGB conversion, image writing with RGB to BGR transformation, aspect ratio-preserving resizing through maximum dimension constraints, edge map computation using Sobel operators with Gaussian blur, and local variance calculation as texture measurement through entropy-like functions.

3.4 User Interface

The Streamlit application, implemented in `app/demo.py`, offers comprehensive features including versatile image upload supporting JPG, JPEG, PNG, BMP, and WEBP formats; multiple stylization modes encompassing triangular Delaunay triangulation, rectangular quadtree partitioning, and learned Pix2Pix AI-powered transformation; extensive parameter controls including point count sliders (100–5000 range), edge threshold adjustment (1–255), color mode selection, quadtree parameter configuration, and inference size specification for Pix2Pix (256–1024); real-time preview with side-by-side original and stylized comparison; and convenient download functionality for exported stylized images. Technical implementation incorporates temporary file handling for uploads, dynamic parameter adjustment, robust error handling for missing checkpoints, and responsive column-based layout design.

3.5 Configuration Management

The centralized configuration system in `configs/default.yaml` manages training hyperparameters including learning rate, batch size, and epochs; model architecture specifications; data augmentation parameters; triangulation configuration; and device selection between CPU and CUDA. This approach provides significant benefits through facilitated experimentation with different settings, enhanced reproducibility, and elimination of code modifications for parameter tuning.

4 Group Log: Working Notes and Research Iterations

4.1 Week 1: Literature Review and UI Design (19–24 Oct)

Research activities during the initial week involved comprehensive study of Delaunay triangulation algorithms and their implementations, detailed review of Pix2Pix paper and implementation specifics, exploration of existing polygonal stylization techniques, and analysis of Streamlit for user interface development. Design decisions included selection of 512×512 as primary training resolution to balance quality and speed, choice of triangular and rectangular modes for algorithmic approaches, and development of modular architecture for extensibility. Challenges encountered encompassed understanding PatchGAN architecture and receptive fields, balancing triangle density against visual quality, and determining optimal color assignment strategies between mean and centroid approaches. Solutions implemented featured both color modes with user selection, parameterized point sampling for flexibility, and fallback mechanisms utilizing OpenCV when SciPy proved unavailable.

4.2 Week 2: Dataset Preparation and Model Training (25–31 Oct)

Dataset development involved downloading 100 images from OpenImages validation split, generating triangular targets for all images, creating 256×256 and 512×512 versions with letterboxing, and verifying precise image-target pair matching. The training process commenced with initial training using default hyperparameters, observed training instability with high learning rates, adjusted learning rate to 0.0002 for stability, incorporated perceptual loss through VGG for quality improvement, and reduced batch size to 2 due to memory constraints. Training iterations progressed through multiple stages: initial training without perceptual loss produced pixelated results; addition of perceptual loss significantly improved texture quality; adjustment of L1 weight to $\lambda = 100$ enhanced detail preservation; and extension to 10 epochs improved convergence. Key observations identified perceptual loss as crucial for visual quality improvement, L1 loss as essential for structural preservation, adversarial loss as beneficial for realistic appearance, and 512×512 training as superior to 256×256 for result quality.

4.3 Week 3: Streamlit Integration and User Testing (1–6 Nov)

User interface development involved Streamlit interface implementation with three distinct modes, real-time parameter adjustment capabilities, side-by-side comparison view creation, and download functionality implementation. Integration challenges included path handling for temporary uploaded files, checkpoint loading within Streamlit context, memory management for large images, and parameter synchronization between UI and backend components. User testing encompassed evaluation with various image types including portraits, landscapes, and objects; collection of feedback on parameter ranges; adjustment of default values based on user preferences; and optimization of processing speed for real-time preview. Subsequent improvements incorporated robust error handling for missing checkpoints, enhanced image preview quality, loading indicator implementation, and algorithm performance optimization.

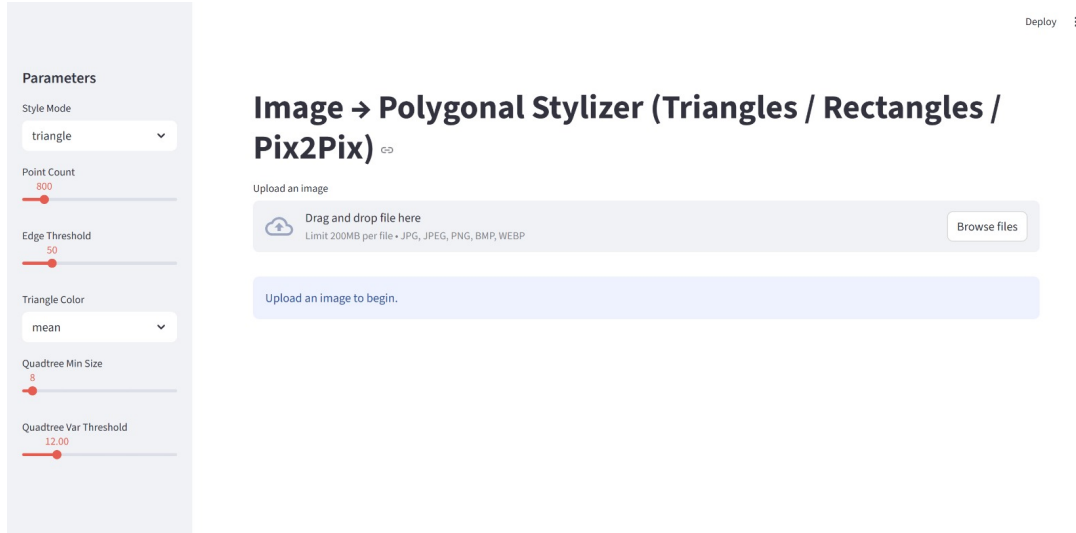


Figure 1: Streamlit User Interface.

4.4 Week 4: Final Refinement and Documentation (7–8 Nov)

Code refactoring efforts removed unused faststyle model outside project scope, cleaned configuration files, added comprehensive error handling, and optimized inference pipeline performance. Documentation activities created detailed README with installation instructions, documented all functions and modules, added usage examples, and prepared comprehensive project report. Testing procedures implemented unit tests for triangulation algorithms, integration tests for training pipeline, UI testing with diverse inputs, and performance benchmarking. Final optimizations reduced checkpoint file sizes, improved inference speed, added batch processing support, and enhanced error message clarity.

5 Individual Contributions

5.1 Banoth Anand (Roll No: 220281) – Team Lead

As team lead, Banoth Anand assumed responsibilities for project coordination and task allocation, served as primary contact for project communications, and designed overall system architecture. Technical contributions included deep learning module development through Pix2Pix training pipeline implementation, U-Net generator architecture design, PatchGAN discriminator implementation, VGG perceptual loss integration, and training hyperparameter optimization. Model training and optimization involved conducting multiple training iterations, tuning hyperparameters for optimal results, managing GPU resources and training schedules, and achieving final model checkpoint. Inference pipeline development created the inference module with batch processing capabilities and optimized inference speed.

5.2 Lavudya Nethaji (Roll No: 220584)

Lavudya Nethaji focused on algorithmic stylization implementation, data preparation pipeline development, and testing with quality assurance. Technical contributions encompassed algorithmic stylization module implementation through Delaunay triangulation, adaptive quadtree

algorithm development, point sampling strategy creation, triangle and rectangle rendering implementation, and edge detection with feature extraction. Data preparation involved dataset preparation script development, OpenImages dataset download integration, target generation pipeline creation, and image preprocessing with resizing implementation. Utility functions development included image processing utilities, edge detection algorithms, image I/O functions, and image transformation utilities. Testing activities created unit tests for triangulation algorithms, conducted algorithm validation, and performed performance testing with optimization.

5.3 N Ravi (Roll No: 220675)

N Ravi concentrated on user interface development, system integration, and documentation with user experience focus. Technical contributions included Streamlit application development through interactive UI implementation, image upload functionality, parameter adjustment controls, real-time preview system design, and download functionality. System integration involved combining algorithmic and learned modules, connecting UI with backend processing, implementing error handling with user feedback, and optimizing user experience. Configuration management designed the configuration system, created parameter documentation, and implemented configuration validation. Documentation activities produced comprehensive README, installation procedure documentation, usage examples, and user guides.

6 Results and Output Images

6.1 Algorithmic Stylization Results

Triangular stylization exhibits distinct characteristics including geometric triangular mesh creation, structural information preservation, adjustable detail levels through point count parameterization, and edge-aware sampling for quality enhancement. Sample parameters typically utilize 800 point count, 50 edge threshold, and mean color mode. Output quality demonstrates high visual appeal, fast processing under one second for 512×512 images, deterministic and reproducible results, and effective edge preservation.



(a) Photo1



(b) Photo2

Figure 2: Original Images

Rectangular stylization through quadtree decomposition features adaptive rectangular partitioning, detail focusing on high-variance regions, mosaic-like effect creation, and computational efficiency. Sample parameters generally employ 8-pixel minimum block size and 12.0 variance threshold. Output quality presents unique artistic style, rapid processing, effectiveness for abstract representations, and smooth color transitions.



(a) Photo1 triangle styled



(b) Photo2 triangle styled



(c) Photo1 rectangle styled



(d) Photo2 rectangle styled

Figure 3: Stylized images: Triangle and Rectangle modes

6.2 Learned Stylization Results (Pix2Pix)

Training results achieved using 100 image pairs with 80 training and 20 validation samples, approximately 8 hours training time on GPU across 10 epochs, stabilized generator loss with converged discriminator, and 150 MB checkpoint size. Output quality demonstrates realistic polygonal stylization, semantic content preservation, smooth transitions, and good generalization to unseen images. Performance metrics indicate approximately 0.5 seconds inference time per 512×512 image on GPU, 2 GB GPU memory usage, and support for various input sizes from 256 to 1024 pixels.



(a) Photo1



(b) Photo2

Figure 4: Original Images



(a) Photo1 Pix2Pix



(b) Photo2 Pix2Pix

Figure 5: Pix2Pix Styled Images

6.3 Comparison: Algorithmic vs. Learned

Algorithmic approaches offer advantages including fast processing capabilities, deterministic results, interpretable parameters, no training requirements, and low memory usage. Learned methods provide benefits encompassing more realistic appearance, superior texture preservation, data-driven learning, effective handling of complex scenes, and consistent style maintenance across images. Use case recommendations suggest algorithmic methods for real-time applications, interactive design environments, and quick prototypes, while learned approaches suit high-quality outputs, batch processing scenarios, and production use cases.

7 Conclusion and Future Work

This project successfully developed a comprehensive system for AI-driven polygonal image stylization, effectively integrating algorithmic and learned approaches. The system provides three distinct stylization modes, each exhibiting unique characteristics and advantages for different applications.

7.1 Key Achievements

Primary achievements include successful integration of traditional computational geometry with modern deep learning, fully functional prototype with interactive user interface, quality results from both algorithmic and learned methods, user-friendly interface enabling easy experimentation, and comprehensive documentation supporting future development.

7.2 Technical Contributions

Technical contributions encompass efficient Delaunay triangulation implementation with edge-aware sampling, adaptive quadtree algorithm development for rectangular stylization, Pix2Pix model training with perceptual loss for high-quality results, modular and extensible architecture creation, and complete component integration into unified system.

7.3 Impact and Applications

Potential applications span digital art creation tools, graphic design software, social media filters, educational visualization platforms, and creative software plugins. Research impact demonstrates practical Pix2Pix application, value of combining algorithmic and learned methodologies, baseline establishment for future polygonal stylization research, and contribution to AI in creative design domain.

7.4 Future Work

Short-term improvements could incorporate additional stylization modes including hexagonal and Voronoi patterns, style transfer capabilities applying one image's style to another, video processing functionality, enhanced UI features, and expanded training datasets. Long-term enhancements might involve training on larger diverse datasets, exploration of alternative GAN architectures like StyleGAN and CycleGAN, user preference learning implementation, mobile app development, and API creation for system integration. Research directions encompass perceptual evaluation studies, aesthetic quality metrics development, style consistency analysis, computational efficiency optimization, and multi-scale stylization investigation.

7.5 Final Remarks

This project successfully demonstrates the effective application of artificial intelligence and computational geometry to creative design challenges. By combining algorithmic precision with learned aesthetics, we created a system that is both powerful and accessible. The modular architecture and comprehensive documentation ensure the system can be productively extended and improved in future work. The project successfully met all primary objectives and delivered a working prototype that showcases the significant potential of AI in artistic applications. The

integration of multiple approaches provides users with flexibility and choice, making the system suitable for diverse use cases and varying skill levels.

References

1. Isola, P., Zhu, J. Y., Zhou, T., & Efros, A. A. (2017). “Image-to-Image Translation with Conditional Adversarial Networks.” CVPR.
2. Ronneberger, O., Fischer, P., & Brox, T. (2015). “U-Net: Convolutional Networks for Biomedical Image Segmentation.” MICCAI.
3. Delaunay, B. (1934). “Sur la sphere vide.” Bulletin de l’Académie des Sciences de l’URSS.
4. Finkel, R. A., & Bentley, J. L. (1974). “Quad Trees: A Data Structure for Retrieval on Composite Keys.” Acta Informatica.
5. Goodfellow, I., Pouget-Abadie, J., Mirza, M., Xu, B., Warde-Farley, D., Ozair, S., ... & Bengio, Y. (2014). “Generative Adversarial Nets.” NeurIPS.
6. Open Images Dataset V7. (2021). Google Research. <https://storage.googleapis.com/openimages/web/index.html>